University Management System
Analysis and Design Document : Assignment 1

Student: Mascasan Maya-Lorena
Group: 30431

Table of Contents

Contents

1. Requirements Analysis 3
- 1.1 Assignment Specification 3
- 1.2 Functional Requirements 3
- 1.3 Non-functional Requirements 3

2. Use-Case Model 4

3. System Architectural Design 5

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to create a web-based application for managing university infrastructure, specifically focusing on the relationships between students, professors, courses, and their respective departments.

1.2 Functional Requirements

The application has the following functional requirements:

1. Allow users to create new person entries (students or professors) with a name, email, age, password, and a dynamic role.
2. Allow users to view all existing person records in a centralized list.
3. Allow users to enroll or assign a person to specific courses by providing a list of course identifiers during the creation process.
4. Allow users to create and manage academic departments and link them to courses to maintain organizational hierarchy.
5. Allow users to delete existing records to keep the university database up to date

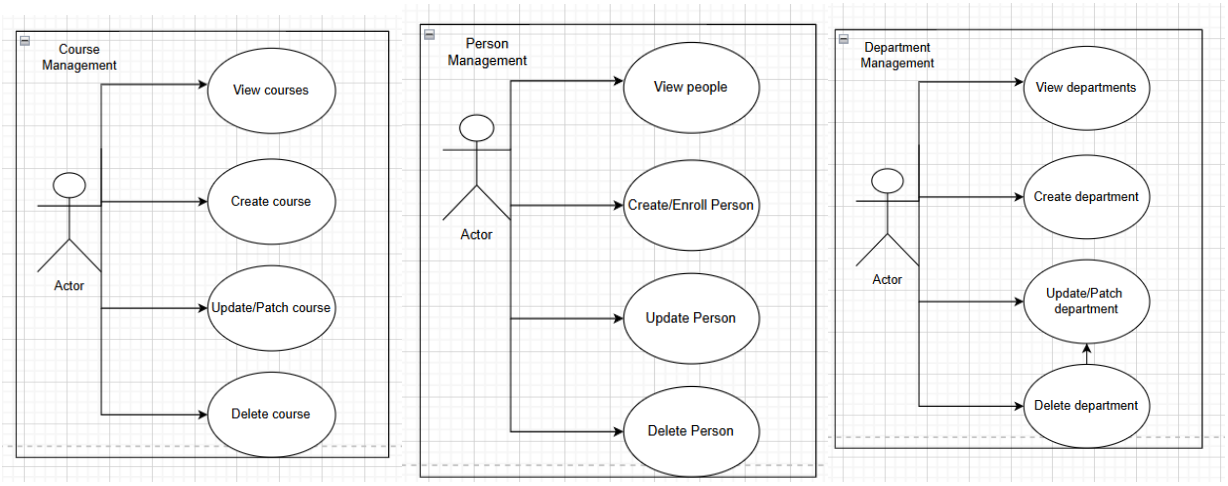
1.3 Non-functional Requirements

The application has the following non-functional requirements:

1. Implement strict server-side validation to ensure that all inputs, such as email formats and password strength, are valid and adhere to the specified format.
2. Use an ORM (Object-Relational Mapping) to handle all database interactions and entity relationships.
3. Use a DI (Dependency Injection) container for managing the lifecycle of services, controllers, and repositories.
4. Use a layered architecture (Controller, Service, Repository) for better organization and separation of concerns.
5. Use a relational database to store all the data persistently.
6. Implement a 3-tier system architectural design consisting of a Presentation Tier, Logic Tier, and Data Tier.

©UTCN, 2026

2. Use-Case Model

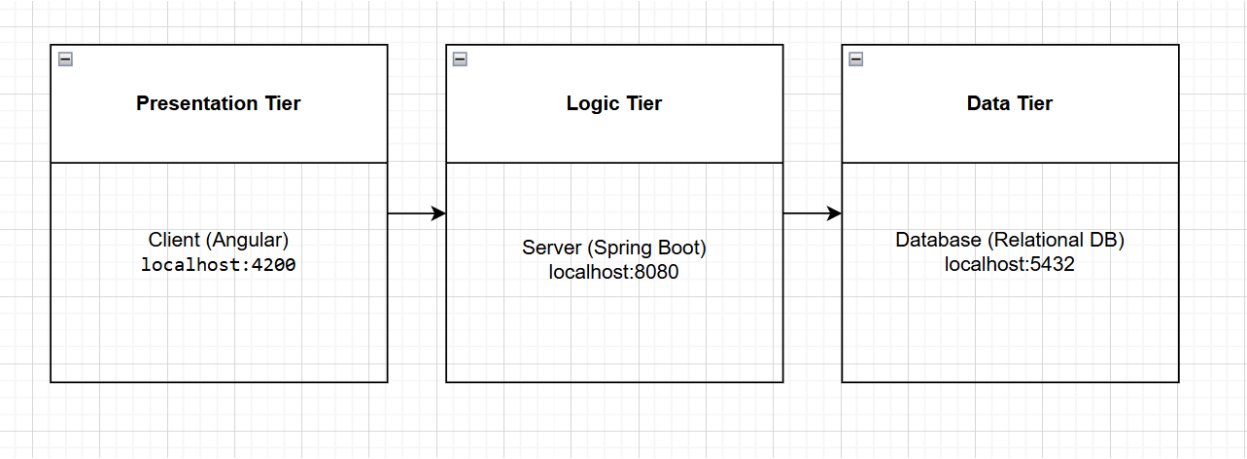


Description of the "View people" use-case:

- **Use case goal:** Fetching all of the recent person records and displaying them on screen using an intuitive table representation.
- **Primary actor:** The user of the application.
- **Main success scenario:** Database connection succeeds, person data is fetched correctly from the logic tier, and is displayed in an intuitive manner within the management dashboard.
- **Extensions:**
    - **Alternate scenario of success:** Database connection succeeds and data is fetched correctly, but no person records are contained in the response due to the database being empty; the management screen displays a message suggesting that the user should create their first person entry.
    - **Failure:** Either the database connection fails, the request times out, the data is corrupt, or the application crashes unexpectedly. The user will get a corresponding error message in the frontend.

3. System Architectural Design

3.1 Architectural Pattern Description

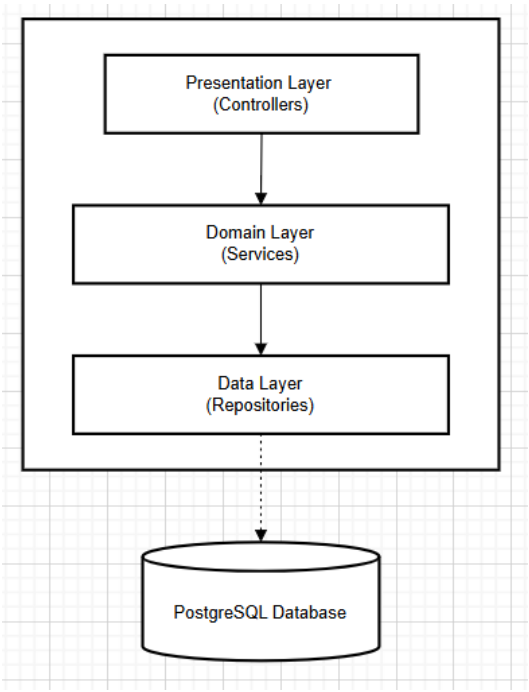


The application follows a **3-tier architectural pattern**, separating the concerns of the user interface, business logic, and data storage.

- **Presentation Tier (Client):** This is the user-facing layer implemented using the **Angular** framework. It runs on the client's browser (typically at http://localhost:4200) and communicates with the logic tier via RESTful API calls using the HttpClient.
- **Logic Tier (Server):** This is the middle layer implemented using **Spring Boot**. It serves as the "Server" (running at http://localhost:8080) where all business rules, such as duplicate email validation or department-course linking, are processed.
- **Data Tier (Database):** This is the persistent storage layer using a **PostgreSQL** relational database. It stores the entities defined in the logic tier, such as Person, Course, and Department.

3.2 Diagrams

And architecture should be layered:

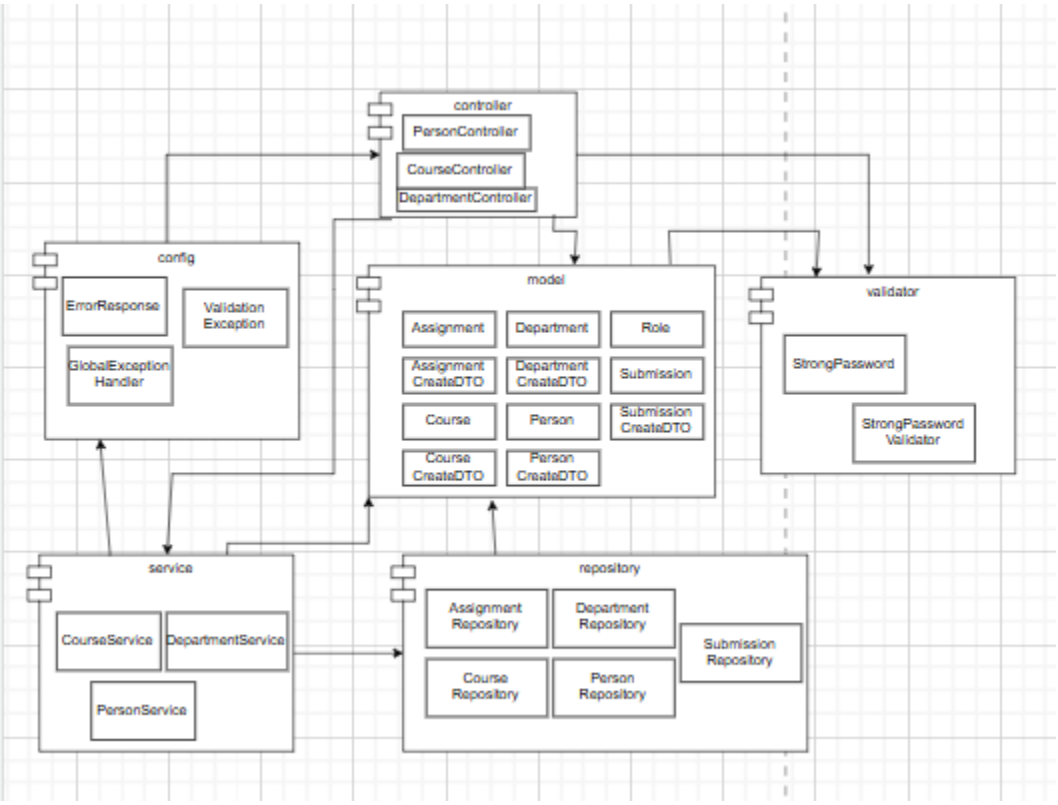


Layered Architecture
In addition to the 3-tier structure, the **Logic Tier** itself is organized into a **layered architecture**:

- **Controller Layer:** The entry point for requests (e.g., PersonController).
- **Service Layer (Domain):** Where the business logic resides (e.g., DepartmentService, CourseService).
- **Repository Layer (Data):** Handles the actual communication with the database using Spring Data JPA.

4. Class Design

4.1 Package + Class Diagram



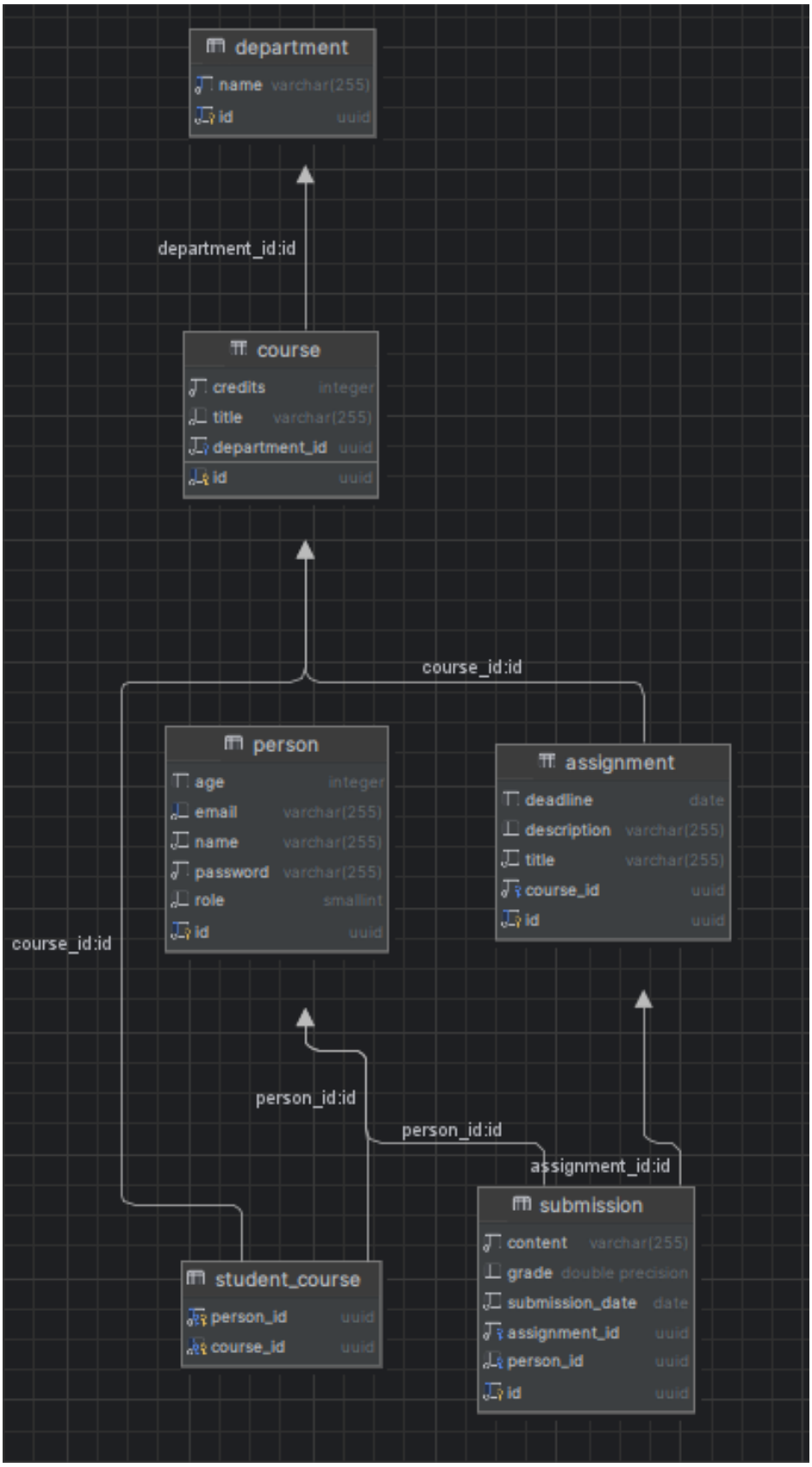
The application's backend architecture is organized into several key modules that enforce a clean separation of concerns. The diagram contains the high-level classes and modules as follows:

- **Controller Layer**: This package contains the REST Controllers (PersonController, CourseController, DepartmentController) which act as the entry point for requests from the presentation tier.
- **Service Layer**: This module contains the business logic implementations (PersonService, CourseService, DepartmentService). It acts as an intermediary, processing data before it is sent to the repository.
- **Repository Layer**: This module manages data access through interfaces such as PersonRepository and CourseRepository, which interact directly with the database.
- **Model Layer**: This central package defines the domain entities (e.g., Person, Course, Department) and the Data Transfer Objects (DTOs) used for creating and updating records.
- **Config & Validator**:
    - **Config**: Contains global handlers like GlobalExceptionHandler to manage application-wide error responses.
    - **Validator**: Contains custom validation logic, such as StrongPasswordValidator, to ensure data integrity before persistence.

5. Data Model

The application's data model is designed to manage complex university relationships through a normalized relational structure:

- **Person Entity**: This represents the central user of the system. It includes core attributes like name, email, age, and password, along with a Role.
- **Department Entity**: Acts as the primary organizational unit. It maintains a **One-to-Many** relationship with the Course entity, ensuring every course belongs to exactly one department.
- **Course Entity**: Stores academic details like title and credits. It maintains a **Many-to-Many** relationship with the Person entity, managed through a join table (visible in the diagram as enrolled_courses), allowing students and professors to be associated with multiple courses.
- **Assignment & Submission Entities**: These entities allow for the creation of academic tasks within a specific Course and track the work submitted by a Person.


department
name varchar(255)
id uuid
department_id:id
course
credits integer
title varchar(255)
department_id uuid
id uuid
course_id:id
person
age integer
email varchar(255)
name varchar(255)
password varchar(255)
role smallint
id uuid
assignment
deadline date
description varchar(255)
title varchar(255)
course_id uuid
id uuid
course_id:id
person_id:id
person_id:id
assignment_id:id
submission
content varchar(255)
grade double precision
submission_date date
assignment_id uuid
person_id uuid
id uuid
student_course
person_id uuid
course_id uuid